GRANAÊ --- DOCUMENTO MESTRE DO PRODUTO

Versão: 1.0\

Data: 08/10/2026\

Status: documento de transferência para outra IA/agente de desenvolvimento\

Nome: Granaê\

Slogan: Sua grana. Seu jogo. Sua evolução.\

Campanha: Aqui, quem joga é você. Quem ganha é a sua vida financeira.

---

1. Visão geral

O Granaê é um aplicativo de controle e educação financeira com gamificação. A proposta é substituir planilhas e controles financeiros complexos por uma experiência simples, visual e motivadora.

A regra central é:

> O usuário melhora sua vida financeira e, como consequência, evolui dentro do jogo.

A gamificação deve incentivar comportamentos financeiros saudáveis. O produto não deve incentivar apostas, cassino, gastos artificiais, endividamento, competição financeira prejudicial ou uso irresponsável da reserva.

---

2. Identidade

Nome: Granaê

Slogan: Sua grana. Seu jogo. Sua evolução.

A identidade visual utiliza principalmente azul, roxo, gradientes e elementos verde/água, com detalhes em laranja em algumas interfaces. A logo oficial contém o nome "granaê" e um símbolo de crescimento/evolução.

---

3. Público e proposta

Público inicialmente considerado: aproximadamente 17 a 46 anos, especialmente pessoas que gostam de tecnologia e jogos e querem organizar melhor a vida financeira.

O produto deve transformar organização financeira em uma jornada de evolução, sem transformar dinheiro real em elemento de aposta.

---

4. Núcleo financeiro

4.1 Contas

O usuário pode cadastrar contas financeiras.

Campos principais:

• nome;

• tipo;

• saldo atual;

• participação na Reserva Financeira;

• descrição opcional.

Valores financeiros são armazenados internamente em inteiros de centavos.

A interface deve formatar valores brasileiros. Exemplo:

• `1000` → `1.000`

• `10000` → `10.000`

4.2 Transações

O usuário registra:

• receitas;

• gastos;

• ajustes.

Campos principais:

• conta;

• categoria;

• tipo;

• valor;

• data;

• observação.

Uma movimentação válida pode gerar evento de gamificação.

Abrir telas ou navegar não gera recompensa.

4.3 Reserva Financeira

Reserva é separada de patrimônio.

Podem participar da Reserva:

• dinheiro destinado à reserva;

• investimentos elegíveis;

• valores líquidos explicitamente classificados como reserva.

Não entram automaticamente:

• imóveis;

• veículos;

• bens físicos;

• patrimônio não líquido;

• dívidas/passivos.

A Reserva não deve ser tratada como dinheiro disponível para realizar

Sonhos.

4.4 Patrimônio

Patrimônio e Reserva são métricas separadas.

Um imóvel, veículo ou outro bem não deve ser interpretado

automaticamente como dinheiro disponível para um Sonho.

4.5 Classe Financeira

A Classe Financeira representa o estado atual baseado na Reserva.

Ela pode subir ou cair.

O histórico de classes permanece.

A Classe não determina diretamente a dificuldade de um Sonho.

---

5. Dashboard

O Dashboard deve mostrar de forma simples:

• nível;

• XP;

• Graninha;

• Streak;

• Reserva Financeira;

• Patrimônio;

• Classe Financeira;

• transações recentes;

• evolução financeira;

• objetivos relevantes.

Dashboard e Perfil utilizam `player_progress` como fonte de verdade para

XP, nível e Graninha.

---

6. Metas e Sonhos

O usuário pode criar:

• Meta;

• Sonho.

Criar um objetivo não gera recompensa.

Marcos oficiais:

• 10%;

• 25%;

• 50%;

• 75%;

• 100%.

Marcos já recebidos permanecem registrados.

Se um aporte atravessar vários marcos, todos os marcos ainda não recebidos podem ser processados uma vez.

Meta

Meta utiliza sua tabela própria de recompensas.

Sonho

Sonho possui analisador de dificuldade.

Criar Sonho não recompensa.

A gamificação do Sonho começa a partir de 10% de progresso.

---

7. Analisador de Dificuldade --- `difficulty_formula_v1`

A dificuldade representa o esforço financeiro necessário para alcançar o valor restante de um Sonho em relação à capacidade sustentável de economia do usuário.

Categorias oficiais:

1. Fácil
2. Moderada
3. Difícil
4. Muito difícil
5. Lendária

7.1 Valor restante

```
valor restante = valor total - progresso acumulado
```

Nunca permitir valor restante negativo.

7.2 Capacidade sustentável

Quando houver histórico suficiente, analisar os últimos 6 meses

disponíveis.

Para cada mês:

```
saldo mensal = receitas - gastos
```

Utilizar os saldos mensais positivos e calcular a mediana.

Depois:

```
capacidade sustentável = mediana dos saldos mensais positivos × 0,70
```

Não utilizar apenas um mês.

Não usar salário bruto como capacidade.

Não usar patrimônio.

Não usar a Reserva Financeira como dinheiro disponível para o Sonho.

7.3 Dados insuficientes

Se não houver histórico suficiente:

• não inventar capacidade;

• não assumir capacidade zero como verdade;

• não classificar automaticamente como fácil.

Mostrar estado equivalente a:

"Dados insuficientes"

Mensagem sugerida:

"Estamos conhecendo seu ritmo financeiro. Continue registrando suas

movimentações para analisarmos melhor a dificuldade deste sonho."

7.4 Confiança

Cada avaliação possui:

• baixa;

• média;

• alta.

A confiança é independente da dificuldade e representa a qualidade dos

dados.

7.5 Sonho com prazo

Calcular:

```
necessidade mensal = valor restante ÷ meses restantes
```

Depois:

```
índice de esforço = necessidade mensal ÷ capacidade sustentável
```

Tabela oficial

Índice Dificuldade

---------- ----------

até 0,50 Fácil

\> 0,50 até 0,75 Moderada

\> 0,75 até 1,00 Difícil

\> 1,00 até 1,50 Muito difícil

\> 1,50 Lendária

7.6 Sonho sem prazo

Não inventar prazo.

Calcular:

```
tempo estimado = valor restante ÷ capacidade sustentável
```

Tabela oficial

Tempo estimado Dificuldade

---------- ----------

até 6 meses Fácil

\> 6 até 12 meses Moderada

\> 12 até 24 meses Difícil

\> 24 até 48 meses Muito difícil

\> 48 meses Lendária

7.7 Dificuldade dinâmica

A dificuldade pode mudar conforme:

• progresso;

• valor restante;

• capacidade financeira;

• prazo;

• novos dados.

Exemplo: um Sonho pode começar como Lendário e depois se tornar Muito difícil.

7.8 Histórico

Utilizar `difficulty_assessments` com:

• id;

• goal_id;

• user_id;

• difficulty_code;

• inputs_json;

• result_json;

• confidence_level;

• formula_version;

• calculated_at.

Não apagar avaliações antigas.

Versão atual:

`difficulty_formula_v1`

---

8. Relação entre dificuldade e raridade

Futuramente:

Dificuldade Raridade

---------- ----------

Fácil Comum

Moderada Incomum

Difícil Rara

Muito difícil Extra Rara

Lendária Lendária

O usuário nunca escolhe a raridade.

A raridade é consequência do resultado do sistema.

---

9. XP

XP representa evolução do jogador.

Características:

• calculado no backend;

• registrado em livro-razão;

• determina o nível;

• não pode ser comprado;

• não pode ser editado pelo frontend.

Nível:

1 a 100

A curva de XP é configurável e versionada.

---

10. Graninha

Graninha é moeda virtual interna.

Regras:

• não é dinheiro real;

• não é conversível;

• não é transferível;

• não compra XP;

• não fornece vantagem financeira;

• não depende diretamente de salário, patrimônio ou saldo;

• pode futuramente comprar itens cosméticos.

---

11. Reward Ledger

`reward_ledger` é imutável.

Cada recompensa registra:

• user_id;

• event_id;

• reward_type;

• amount;

• reason_code;

• config_version;

• created_at.

Correções usam lançamentos de ajuste auditáveis.

Nunca editar silenciosamente uma recompensa histórica.

---

12. Player Progress

`player_progress` contém a projeção atual:

• user_id;

• xp_total;

• level;

• graninha_balance;

• updated_at.

Dashboard e Perfil devem ler essa entidade.

O cliente não pode alterar esses campos diretamente.

---

13. Eventos de gamificação

Eventos podem incluir:

• `expense_recorded`;

• `income_recorded`;

• `financial_entry_updated`;

• `financial_entry_archived`;

• `goal_progress_added`.

Eventos futuros, como `no_expense_day` e `weekly_review_completed`,

ficam preparados até suas regras serem aprovadas.

Cada evento possui:

• id;

• user_id;

• event_type;

• occurred_at;

• business_date;

• idempotency_key;

• entity_type;

• entity_id;

• payload;

• processed.

---

14. Processamento de gamificação

Fluxo:

1. usuário realiza ação;

2. backend valida;

3. ação é persistida;

4. evento é criado;

5. motor verifica elegibilidade;

6. recompensa é registrada, se aplicável;

7. `player_progress` é atualizado;

8. nível é recalculado.

A lógica crítica fica no backend.

O frontend apenas exibe.

Se o motor falhar após a ação financeira:

• a ação financeira continua confirmada;

• evento permanece `processed=false`;

• o evento pode ser processado posteriormente;

• retries não podem duplicar recompensa.

---

15. Idempotência

Cada evento possui `idempotency_key`.

A mesma operação repetida não pode gerar recompensa duplicada.

O ledger deve impedir duplicidade por:

• event_id;

• reason_code;

• reward_type.

---

16. Streak

Uma data é válida quando existe pelo menos uma ação financeira válida naquele dia.

Navegação não conta.

Primeira ação do dia:

• atualiza o streak.

Outras ações no mesmo dia:

• não aumentam novamente.

Se última data foi ontem:

• incrementa.

Se foi hoje:

• não altera.

Se houver lacuna:

• inicia novo ciclo com 1.

Marcos:

• 3;

• 7;

• 14;

• 30;

• 60;

• 100;

• 365 dias.

Quebrar streak não remove recompensas passadas.

Não existe proteção de streak na v1.0.

---

17. Hábitos

O sistema possui hábitos financeiros.

As recompensas por hábito possuem limites por período.

Vários lançamentos no mesmo período não podem gerar várias recompensas do mesmo hábito.

---

18. Missões

Missões são controladas pelo servidor.

Possíveis dificuldades:

• fácil;

• média;

• difícil;

• especial.

Missões não devem incentivar gasto, dívida, aposta ou comportamento financeiro artificial.

---

19. Conquistas

Plano inicial:

30 conquistas

Cada conquista:

• possui condição;

• pode possuir raridade;

• é desbloqueada uma vez por usuário;

• possui condição versionada;

• é avaliada no backend.

Condições qualitativas não devem ser inventadas.

---

20. Selos

Plano inicial:

10 selos

Representam hábitos, identidade e evolução.

Podem possuir níveis.

---

21. Troféus

Plano inicial:

5 troféus

Representam feitos especiais.

São únicos por usuário.

Não devem servir como fonte infinita de XP ou Graninha.

---

22. Itens e raridades

Raridades:

• Padrão;

• Raro;

• Extra Raro;

• Mítico;

• Lendário.

Itens são estritamente cosméticos.

Categorias:

• tema;

• fundo;

• moldura;

• ícone;

• efeito;

• animação;

• avatar;

• futuros itens do mascote.

Itens nunca podem possuir:

• XP multiplier;

• reward bonus;

• vantagem financeira;

• vantagem em Sonhos;

• vantagem em dificuldade.

---

23. Mascote

Mascote:

Tio Gansinhas

Descrição:

• ganso antropomórfico branco;

• óculos escuros;

• corrente dourada;

• jaqueta bomber preta com detalhes laranja;

• tênis.

Animações avançadas ficam fora do núcleo inicial.

---

24. Segurança e isolamento

Cada usuário só pode acessar seus próprios dados.

Aplicar isolamento em:

• perfil;

• contas;

• transações;

• objetivos;

• avaliações;

• eventos;

• ledger;

• progresso;

• streak;

• conquistas;

• selos;

• troféus;

• inventário.

RLS ou mecanismo equivalente deve garantir:

```
user_id = usuário autenticado
```

Configurações econômicas possuem acesso administrativo para escrita.

---

25. Anti-exploit

Proteger contra:

• repetição de requisição;

• duplicação de recompensa;

• criação artificial de lançamentos;

• alteração de relógio;

• manipulação de progresso;

• alteração manual de dificuldade;

• falsificação de XP;

• falsificação de Graninha;

• compra duplicada de item;

• manipulação de raridade.

---

26. Auditoria

Operações sensíveis registram:

• ator;

• ação;

• entidade;

• identificador;

• request_id;

• data;

• metadados mínimos.

Não enviar lançamentos financeiros completos para telemetria externa.

---

27. Versionamento econômico

Utilizar `economy_config_versions`.

Cada configuração possui:

• versão;

• status;

• vigência;

• parâmetros;

• origem.

Status:

• draft;

• active;

• retired.

Alterar uma configuração não altera recompensas históricas.

---

28. Modelo de dados

Identidade

• `profiles`

• `user_consents`

• `user_devices`

Financeiro

• `financial_accounts`

• `financial_categories`

• `financial_entries`

• `net_worth_snapshots`

• `reserve_snapshots`

• `financial_class_history`

Objetivos

• `goals`

• `goal_progress_entries`

• `goal_milestone_claims`

• `difficulty_assessments`

Gamificação

• `gamification_events`

• `reward_ledger`

- player_progress
- habit_completions
- streaks
- streak_milestone_claims
- economy_config_versions

Conteúdo

- missions
- user_missions
- achievement_definitions
- user_achievements
- badge_definitions
- user_badges
- trophy_definitions
- user_trophies

Cosméticos

- cosmetic_items
- inventory_items
- equipped_cosmetics

Segurança

- audit_logs
- security_events
- outbox_events

---

29. Arquitetura técnica

Arquitetura lógica:

```
Usuário<br/> ↓<br/>Aplicativo<br/> ↓<br/>API autenticada<br/> ↓<br/>Serviços de domínio<br/> ↓<br/>Motor de regras<br/> ↓<br/>Banco de dados
```

Stack originalmente planejada:

- Expo;
- React Native;
- TypeScript;
- Supabase;
- PostgreSQL;
- Supabase Auth;
- Edge Functions;
- SQL/RPC;

• TanStack Query;

• Zustand;

• React Hook Form;

• Zod;

• Sentry;

• Vitest;

• React Native Testing Library;

• Playwright.

O desenvolvimento atual está sendo feito com **Base44**.

A plataforma pode mudar, mas as regras de domínio deste documento

permanecem como fonte de verdade.

---

30. Valores monetários

Sempre armazenar dinheiro como inteiro em centavos.

Exemplo:

R\$ 10,50 → `1050`

Nunca usar ponto flutuante como fonte de verdade financeira.

---

31. Testes obrigatórios

Testar:

• criação de conta;

• receitas;

• gastos;

• atualização de saldo;

• Reserva;

• Patrimônio;

• Classe;

• RLS;

• eventos;

• idempotência;

• XP;

• Graninha;

• Streak;

• Metas;

• Sonhos;

• marcos;

• dificuldade;

• confiança;

• histórico de avaliações;

• segurança;

• persistência após recarregar.

Para dificuldade, testar especificamente:

• índice 0,50 → Fácil;

• 0,51 → Moderada;

• 0,75 → Moderada;

• 0,76 → Difícil;

• 1,00 → Difícil;

• 1,01 → Muito difícil;

• 1,50 → Muito difícil;

• 1,51 → Lendária.

Para Sonhos sem prazo:

• até 6 meses → Fácil;

• 6 até 12 → Moderada;

• 12 até 24 → Difícil;

• 24 até 48 → Muito difícil;

• 48 → Lendária.

---

32. Roadmap

Fase 1 --- Fundação

Autenticação, perfil e isolamento.

Concluída/aprovada.

Fase 2 --- Núcleo Financeiro

Contas, categorias, transações, patrimônio, reserva e classe.

Concluída/aprovada.

Fase 3 --- Economia

Eventos, XP, Graninha, níveis, ledger e configurações.

Concluída/aprovada.

Fase 4 --- Hábitos e Streak

Hábitos, streak, marcos e limites.

Concluída/aprovada.

Fase 5/6 --- Metas e Sonhos

Objetivos, progresso e marcos.

Concluída/aprovada.

Fase 7 --- Analisador

Dificuldade, capacidade sustentável, confiança e histórico.

Em implementação/validação.

Próximas fases

• 30 conquistas;

• 10 selos;

• 5 troféus;

• itens;

• raridades;

• inventário;

• loja;

• personalização;

• mascote avançado;

• recursos sociais futuros.

---

33. Regras absolutas para qualquer IA que continuar o projeto

Antes de modificar o Granaê:

1. Ler este documento.

2. Não inventar regras de produto.

3. Não alterar regras congeladas sem aprovação.

4. Não modificar XP/Graninha pelo frontend.

5. Não permitir acesso entre usuários.

6. Não alterar histórico econômico silenciosamente.

7. Manter valores em centavos.

8. Manter o ledger imutável.

9. Versionar mudanças econômicas.

10. Não transformar itens em vantagens.

11. Não criar aposta ou cassino.

12. Não incentivar gasto real.

13. Não usar patrimônio como dinheiro disponível para Sonhos.

14. Não usar Reserva Financeira como dinheiro disponível para Sonhos.

15. Não permitir que o usuário escolha dificuldade ou raridade.

16. Testar mudanças antes de considerar uma fase concluída.

17. Não avançar automaticamente para outra fase sem aprovação.

18. Preservar a filosofia central do Granaê.

---

34. Filosofia final

O Granaê não existe para fazer o usuário jogar mais.

Existe para fazer o usuário **melhorar financeiramente enquanto joga**.

A frase que resume o produto é:

> **Aqui, quem joga é você. Quem ganha é a sua vida financeira.**